

Contents

Module 1 — Computers, Operating Systems, and Civil Engineering Programs	1
Learning outcomes	1
100-minute sequence	1
1. Purpose and learning method	1
2. Four broad stages in computer history	2
3. How a computer and its OS work	2
4. Numbers inside a computer	2
5. Programs in civil engineering	3
6. Program and script demonstrations	3
Short exercise — 3 questions	4
Checklist	4
Further reading	4

Module 1 — Computers, Operating Systems, and Civil Engineering Programs

Learning outcomes

Students will be able to:

- describe the aims, scope, learning skills, and learning method of the course;
- map the relationship among input, CPU, memory, storage, output, and the operating system;
- interpret a simple binary number;
- distinguish a program, a script, and an application; and
- justify when to use existing software and when to create a small automation tool.

100-minute sequence

Minutes	Activity
0–10	Course purpose and civil engineering examples
10–22	A concise history of computers
22–38	Computer components and operating systems
38–53	Decimal, binary, bits, and bytes
53–68	Computer programs in civil engineering
68–82	Excel VBA and AutoCAD VBA demonstrations
82–95	Three-question short exercise
95–100	Review and exit ticket

Minimum product: one diagram of how a computer processes work, one *use–adapt–build* decision, and observations from two short scripts.

1. Purpose and learning method

This course develops computational thinking: breaking down a problem, expressing its steps, translating them into instructions, and checking the result. The core learning skills are:

- **decomposition:** divide a large problem into smaller tasks;
- **abstraction:** retain relevant data and assumptions;
- **pattern recognition:** identify repeated calculations;

- **algorithm design:** organise unambiguous steps;
- **debugging:** locate the cause of an incorrect result; and
- **validation:** compare program output with a reference case.

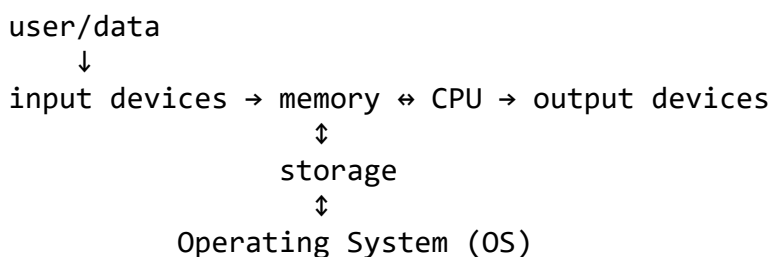
The class uses *predict–run–explain*: predict the output, run the example, and explain any difference between the prediction and actual result.

2. Four broad stages in computer history

Stage	Key development	Consequence
Mechanical	calculating devices and mechanical machines	arithmetic could be assisted
Early electronic	vacuum tubes and machine instructions	complex calculations became possible
Transistors and integrated circuits	smaller, faster, more reliable hardware	computing entered laboratories
Microprocessors and networks	PCs, internet, cloud, and mobile devices	computing became part of everyday life

Across every stage, a computer accepts data, executes instructions, stores state, and produces output.

3. How a computer and its OS work



The CPU executes elementary instructions. Memory holds data currently being used. Storage retains data after the power is turned off. The operating system manages hardware, files, memory, processes, users, and the interface so every application does not need to control hardware directly.

Windows, Linux, macOS, Android, and iOS are operating systems. Excel and AutoCAD are applications that run on an OS. VBA is an automation language and environment hosted inside a particular application.

4. Numbers inside a computer

A **bit** has two states: 0 or 1. Eight bits form one **byte**. Binary digits use powers of two.

Position	3	2	1	0
Weight	8	4	2	1
Digits for 13	1	1	0	1

Therefore, $1101_2 = 8 + 4 + 0 + 1 = 13_{10}$.

Text, images, and fractional numbers are also represented as bit patterns. Some decimal fractions cannot be represented exactly, so computer calculations may contain tiny rounding differences. Engineering programs should compare fractional results using an appropriate tolerance.

5. Programs in civil engineering

Common uses include:

- spreadsheets for quantities, cost, and tabular data;
- CAD/BIM for drawings and information models;
- GIS for spatial data;
- structural, geotechnical, transport, and hydraulic analysis packages; and
- scripts for cleaning data, repeated calculations, reporting, and quality checks.

Use, adapt, or build?

Question	Favors existing software	Favors a small script/t
Is the problem common and already well tested?	yes	no
Is the workflow specialised and repetitive?	sometimes	yes
Is safety risk high?	use validated engineering software	use scripts only as ch
Is development cost greater than the benefit?	yes	no

A practical option is often to use established software and automate only its repetitive parts.

6. Program and script demonstrations

A program is a collection of instructions that completes a task. A script is commonly smaller, runs inside a host environment, and automates a workflow. The boundary is not absolute.

Demo A — Excel VBA

Option Explicit

```
Sub ExcelDemo()  
    Range("A1").Value = "Length (m)"  
    Range("B1").Value = 10  
    Range("A2").Value = "Width (m)"  
    Range("B2").Value = 2  
    Range("A3").Value = "Area (m2)"  
    Range("B3").Value = Range("B1").Value * Range("B2").Value  
End Sub
```

Observe the sequence: write labels, write inputs, read inputs, calculate, and write the output.

Demo B — AutoCAD VBA (optional)

Run this demonstration only where the AutoCAD VBA environment is available. Coordinates follow the active drawing units.

```
Sub CreateSimpleLine()  
    Dim startPoint(0 To 2) As Double  
    Dim endPoint(0 To 2) As Double  
    Dim lineObject As AcadLine  
  
    startPoint(0) = 0: startPoint(1) = 0: startPoint(2) = 0  
    endPoint(0) = 10: endPoint(1) = 5: endPoint(2) = 0
```

```
Set lineObject = ThisDrawing.ModelSpace.AddLine(startPoint, endPoint)
ZoomAll
End Sub
```

The `ThisDrawing.ModelSpace.AddLine` hierarchy follows the official Autodesk ActiveX object hierarchy.

Short exercise — 3 questions

1. Predict

If a user fills B1 and B2, then runs the `ExcelDemo` macro, order the components involved until the result appears in B3. Use these terms: input, Excel, VBA, CPU/memory, worksheet, and output.

2. Make an engineering decision

You must calculate volume and cost for 500 channel segments every week. Choose among existing software, a large custom application, or a small Excel script. Give two reasons and one risk.

3. Explain

In the Excel demo, identify the input, process, output, application, scripting language, and operating system.

Answer key and evidence of understanding

1. A reasonable sequence is: the user provides input through the worksheet, Excel runs the VBA macro, the OS manages the process and memory, the CPU executes the multiplication instructions, and Excel writes the output to B3.
2. A small Excel script is reasonable because the workflow is specialised and repeated. Risks include incorrect formulas/units or overwriting data; mitigate them with tests, validation, and backups.
3. Inputs: B1 and B2; process: multiplication; output: B3; application: Excel; language: VBA; OS: for example Windows, which manages the application and hardware.

Checklist

- ☐ I can map the CPU, memory, storage, and OS.
- ☐ I can convert a four- or five-digit binary number to decimal.
- ☐ I can distinguish an application from a script.
- ☐ I can justify when a small automation tool is appropriate.

Further reading

1. J. Glenn Brookshear & Dennis Brylow, *Computer Science: An Overview*, 13th ed., Pearson, 2019.
2. Andrew S. Tanenbaum & Herbert Bos, *Modern Operating Systems*, 4th ed., Pearson, 2014.
3. Abraham Silberschatz, Peter B. Galvin & Greg Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018.
4. Ronald W. Larsen, *Engineering with Excel*, 5th ed., Pearson, 2017.

← Module list · Module 2 →